

Distributed Hash Table Socket Project

September 27 milestone · Design document

Scope. Implement registration, DHT setup, ring ID assignment, distribution of 1950 storm records, per-peer counts, and completion signaling. This document describes the milestone implementation only; query, join, leave, and teardown are deferred to the full-project deadline.

Deployment

Manager + PeerA (leader)
CloudLab node0 · 10.10.1.1

PeerB + PeerC
CloudLab node1 · 10.10.1.2

Three peer processes across two distinct end hosts, connected over UDP.

Assignment configuration

Socket Project Group 14. The PDF assigns even group 14 ports 8000-8499. This run used 8000-8006.

Manager: 8000
PeerA: 8001 / 8002
PeerB: 8003 / 8004
PeerC: 8005 / 8006

Milestone result

223 storm records read	449 hash table size	SUCCESS DHT completion
--	---	--

The CSV was absent from the course Canvas Files and Socket Programming Project module when checked. The included dataset is a documented NOAA/NWS fallback projection; the assignment does not explicitly say students should derive a missing file this way. See DATA_SOURCE.md.

1 · Architecture and registration

The manager maintains peer metadata and lifecycle state. Each peer uses one UDP socket for manager messages and one UDP socket for peer messages. Multiple peer processes may share a host when their ports are unique.

Message envelope. All messages are compact UTF-8 JSON datagrams. Requests contain type, unique request_id, sender, and a payload object. Responses echo request_id and include status and payload.

Registration message format

```
{"type": "REGISTER", "request_id": "UUID", "sender": "PeerA",  
  "payload": {"ip": "10.10.1.1", "m_port": 8001, "p_port": 8002}}
```

```
{"type": "REGISTER_RESPONSE", "request_id": "UUID", "status": "SUCCESS", "payload": {}}
```

The manager validates an alphabetic peer name of at most 15 characters, unique name, numeric ports in the configured group range, distinct local manager/peer ports, and no port collision for peers sharing an IP. On success it stores the peer as Free. The peer then binds its peer-to-peer UDP socket.

Time-space diagram · register

Time ↓	Peer process	Manager	Local effect
1	REGISTER(name, IP, m-port, p-port) →	Receive UDP datagram	
2		Validate uniqueness, name, and ports; store peer state as Free	Manager state updated
3	← REGISTER_RESPONSE(SUCCESS / FAILURE)	Send correlated response	
4	On SUCCESS, bind p-port and start peer listener		Peer is ready for ring messages

All three peers registered successfully in the CloudLab run. The manager log shows PeerC, PeerB, and PeerA each received SUCCESS.

Data structures

- Manager peer dictionary keyed by peer name, with IP, ports, and state.
- Ordered member list returned at setup; each peer's right neighbor is member $(id + 1) \bmod n$.
- Each peer's local table maps hash position to a nested event-ID/record mapping to handle collisions.
- Per-peer queues correlate STORE acknowledgments and completion of count collection.

2 · DHT setup and ring construction

setup-dht message format

```
{"type":"SETUP_DHT","request_id":"UUID","sender":"PeerA",
  "payload":{"size":3,"year":1950}}
```

```
{"type":"SETUP_DHT_RESPONSE","request_id":"UUID","status":"SUCCESS",
  "payload":{"year":1950,"members":[
    {"name":"PeerA","ip":"10.10.1.1","p_port":8002},
    {"name":"PeerC","ip":"10.10.1.2","p_port":8004},
    {"name":"PeerB","ip":"10.10.1.2","p_port":8006}]}}
```

The manager rejects an invalid size, unregistered leader, too few Free peers, or an already active DHT. It selects two Free peers at random, marks the leader Leader and the others InDHT, and returns the leader's tuple first. IDs follow this returned order; PeerA is ID 0.

Time-space diagram · setup and ID assignment

Time ↓	Leader (ID 0)	Manager	Peer 1	Peer 2
1	SETUP_DHT(size=3, year=1950) →	Validate leader/size/free peers		
2	← SUCCESS + ordered member tuples	Choose 2 Free peers; set Leader/InDHT states		
3	Set ID 0; ring_size=3; calculate right neighbor			
4	SET_ID(id=1, ring_size=3, members) →		Store ID 1 and right neighbor	
5	SET_ID(id=2, ring_size=3, members) →			Store ID 2 and right neighbor

Build messages

STORE carries target ID, position, hop count, full record, and leader address. A non-target peer increments the hop count and forwards **STORE** to its right neighbor. The target stores the record and sends **STORE_ACK** to the leader. The leader sends a record directly to its right neighbor, never directly to the target node.

```
{"type":"STORE","request_id":"UUID","sender":"PeerA",
  "payload":{"target_id":2,"position":317,"hop_count":0,
    "record":{"event_id":"...", "state":"..."},
    "leader":{"ip":"10.10.1.1","p_port":8002}}}
```

The leader stores ID-0 records locally and transfers other records one at a time, waiting for an acknowledgment. After data transfer it initiates a **COUNT** message around the ring.

3 · Data distribution and completion

Hashing

For ℓ records, choose table size s as the first prime larger than 2ℓ . For each record, $\text{pos} = \text{event_id} \bmod s$ and $\text{id} = \text{pos} \bmod n$. This run has $\ell = 223$, so the first prime above 446 is **$s = 449$** . The destination peer ID is 0, 1, or 2.

COUNT exchange. A COUNT message carries a map of ring ID to local record count and a hop counter. Each peer appends its count and forwards to its right neighbor. On its return, the leader adds its local count, prints the complete map, verifies the total equals 223, then sends DHT_COMPLETE to the manager.

Time-space diagram · count and completion

Time ↓	Leader (0)	Peer 1	Peer 2	Manager
1	COUNT(counts={}) →			
2		Add own count; forward COUNT →		
3			Add own count; forward COUNT →	
4	Receive return; add ID-0 count; verify sum=223			
5	DHT_COMPLETE(peer-name) →			Accept only current leader; mark DHT Ready
6	← DHT_COMPLETE_RESPONSE(SUCCESS)			Log counts and successful completion

dht-complete format

```
{"type":"DHT_COMPLETE","request_id":"UUID","sender":"PeerA",  
  "payload":{"counts":{"0":72,"1":75,"2":76}}}
```

The manager accepts this only from the current leader during construction, changes its phase to Ready, logs the counts, and replies SUCCESS.

Verified output

72 ring ID 0 · PeerA	75 ring ID 1 · PeerC	76 ring ID 2 · PeerB
-------------------------	-------------------------	-------------------------

Total = 223 records. CloudLab logs confirmed all three registrations, SETUP SUCCESS, all IDs, record distribution, counts, and DHT_COMPLETE SUCCESS.

4 · Implementation, test evidence, and submission notes

Implementation choices

- Python 3 standard library: socket UDP, threading, queue, json, and csv.
- Manager is an always-on UDP service. Peer command input is read from stdin; peer and manager messages are correlated by UUID request IDs.
- Ports are unique per process. Group 14 uses the range 8000-8499; this run used 8000-8006.
- Scope is restricted to the September milestone. Graceful termination and October features are not claimed.

Version-control history snapshot

Exported from the local Git repository with `git log --oneline`. Every commit has one author, no co-author trailer, and a one-line subject.

521646c	Initialize milestone project
f458e1d	Add UDP message protocol
6ad20f9	Add storm record hashing
fc99683	Implement manager registration
1c85114	Implement peer ring distribution
2487125	Add milestone dataset and tests
64deee1	Document NOAA dataset provenance
ed69fd6	Record CloudLab milestone results
faddb3c	Run tests with standard library
fc6a70c	Add milestone demo checklist

Demo link and timestamps

Not yet recorded. Replace this block in the final PDF with the public YouTube link and timestamps for rubric steps 3(a)-3(d). The required demo must be no longer than 7 minutes, unedited, and include audio.

Suggested timestamp labels: (a) compile manager/peer; (b) show two distinct end hosts; (c) manager plus three registered peers; (d) setup-dht with 3 peers and 1950 data, per-peer counts, and DHT_COMPLETE SUCCESS.

Dataset source note

The included CSV is a 14-field projection of NOAA/NWS's public archive file `StormEvents_details-ftp_v1.0_d1950_c20260323.csv.gz`, downloaded from the NOAA NCEI Storm Events bulk CSV directory. Its mapping and the fact this is a fallback, not an explicit Canvas instruction, are documented in `DATA_SOURCE.md`.

Source: <https://www.ncei.noaa.gov/pub/data/swdi/stormevents/csvfiles/> · Test details: `MILESTONE_TEST.md` · Demo checklist: `DEMO_SCRIPT.md` · Run instructions: `README.md`